

Lab 3 – Architecture Justification

Architecture Selection: We chose Layered Architecture for the Smart Expense & Budget Management System.

1. Architectural Choice

A three-layer (Presentation / Business / Data) architecture, deployed as browser client → application server → database server. The Web User Interface is the only presentation component. The business layer holds an Application Controller plus six service components (Authentication, Transaction Manager, Budget & Goal Manager, Notification, Analytics Engine, Report Exporter). A single Database Component forms the data layer. Each layer calls only the layer beneath it, through the eight interfaces shown in the component diagram.

2. Two Reasons

Reason 1 – The data is tightly linked and must be consistent. Adding one expense has to change the remaining budget, the category charts and the alert status at once (FR03, FR08, FR09, NFR04). With one relational database, a single transaction covers all of that. A microservices design would split this data across services and rely on eventual consistency for a balance the user expects to be correct immediately.

Reason 2 – The features map cleanly onto layers, and the smart features stay replaceable. Every functional requirement belongs to one business component. Forecasting, anomaly detection, category suggestion and the what-if simulator sit together behind the single `IAalyticsService` interface, so the model can be retrained or replaced without touching the UI or any other component. The scale also fits: a small team and the 1000-user target used in the test plan (NFR05) do not justify deploying and monitoring six or more separate services.

3. Security Advantage

All traffic enters through one point, the Application Controller, which checks the session and validates input before any service runs (SR01, SR05). Only the Authentication Service handles credentials, storing hashed passwords and issuing tokens (SR02, SR03). The browser never reaches the database: only the business layer does, over TLS (SR06), so every query can be filtered by the logged-in user's ID in one place and one user cannot read another's records (SR04). One public entry point also means a much smaller attack surface than many exposed services.

4. Performance Benefit

The controller and services run in the same server process, so calls between them are in-process rather than network hops, avoiding the latency cost that microservices add. A dashboard load is one HTTPS round trip from the browser; the services assemble totals, charts and the forecast using indexed queries, which supports the 3-second dashboard target (NFR01). The business tier keeps no state beyond the session token, so extra copies can run behind a load balancer if usage grows.

5. Trade-off and Styles Not Chosen

The weakness of layering is that components cannot scale independently. The Analytics Engine is the most likely to need that, and because it is already isolated behind `IAalyticsService` it could later move into its own service without changing its callers. A plain Client-Server design would be simpler still, but it mixes interface logic, business rules and data access, which makes the modular changes required by NFR06 harder. Microservices would give independent scaling and fault isolation, but the added network latency, data-consistency work and operational effort outweigh those gains at this project's size.